

Car Acceptability Classification Using Multiple Machine Learning Models

Raja Shekhar Dasari
dasari.107@wright.edu | U01112124

Abstract— In this project, I have tested all four classifications on the car Evaluation Dataset: Those are Decision Tree, Random Forest, Logistic Regression and SVM. The problem Statement and the task is to predicting using the categorical features whether a car is unacceptable, Acceptable ,Good or very good. The categorical six features are like buying price ,safety, and the number of doors. Besides, the one-hot encoded features are ended up with the 21 binary columns. Moreover, the data was spitted in to 80/20 with a good sampling to keep the class ratios intact. Also, The Random Forest has got the best accuracy at the 97.11% and the macro F1 of 96.06%.Whereas, SVM had the best balanced accuracy at the 98.13%,and the foremost meaningful metric is here since the 70% of the data belongs to the same class. Likewise the Cross validation on the Random Forest gave the 75.93% when compared to the test accuracy.

Keywords— car evaluation, decision tree, random forest, SVM, logistic regression, one-hot encoding, class imbalance, multiclass classification

GitHub Link: https://github.com/raaj2000/car_evaluation-ml.git

I. INTRODUCTION

Car evaluation is basically a classification task. Moving Further, Given some information about a car is that how expensive it is to buy and also how much it costs to maintain, how many people it seats, and how safe it is and the job is to label it as unacceptable, acceptable, good, or very good. Moreover, The UCI dataset for this problem is clean and fully labeled, which makes it a good testbed for comparing classifiers. Although, one thing that makes it tricky is that the class imbalance: 70% of the 1,728 records are labeled unacceptable. Whereas, that means a model can hit high accuracy just by predicting the majority class, so you need the additional metrics to really tell which model is working well.

I have ran all the four classifiers under the same preprocessing and split conditions to get a fair comparison. However, Here is what this project covers:

- Comparing all four classifiers head-to-head on the same data and split.
- Using three metrics accuracy, balanced accuracy, and macro F1-so class imbalance does not hide weak performance on minority classes.
- Looking at per-class recall to see which classifiers actually handle the good and very good classes, not just unacceptable.

II. RELATED WORK

a. Supervised Learning Approaches

One of the simpler classifiers to use here is the Decision trees. The Quinlan's first research was on ID3 and C4.5, which set the rules for how we divide data recursively based on information gain. The Breiman later improved the Random Forest method. This method builds a lot of trees on a random forest subset of the data and then averages their outputs to avoid overfitting. Both methods also naturally handle categorical features, which is especially good for this dataset.

b. Foundation Models and Transfer Learning

When the class boundaries aren't linear, The SVMs with an RBF kernel tend to work well and comes in to picture . I put logistic Regression in as a baseline because it's easy to understand and use. Although, Neither model needs to be trained ahead of time, which makes sense for a small

dataset like this. Moreover, Transfer learning is also not applicable in this context as it does not involve image or text data.

III. METHODOLOGY

a. Problem Formulation

The challenge involves forecasting one of four class labels (e.g., unacc, acc, good, vgood) based on six categorical inputs. Each sample is a 21-dimensional binary vector after one-hot encoding. The training goal for Logistic Regression is cross-entropy loss with L2 regularization:

$$L(\theta) = \sum \ell(f(\theta(x_i), y_i) + \lambda R(\theta) \quad (1)$$

Here ℓ is cross-entropy, $R(\theta) = \|\theta\|^2$ is L2 regularization, and λ scales the penalty. For which the Decision Tree and Random Forest minimize Gini impurity when splitting nodes. Whereas, SVM finds the max-margin decision boundary using an RBF kernel.

c. Model Architecture

All the four models were built using the scikit-learn. Whereas, the Logistic Regression needed max_iter=2000 because the default 100 iterations was not enough to converge. Besides, The Decision Tree, Random Forest (200 trees), and SVM (RBF kernel) all used class_weight=balanced so the minority classes get more attention during training. The random_state=42 was fixed across all models. Every model saw the same 21 one-hot encoded input features.

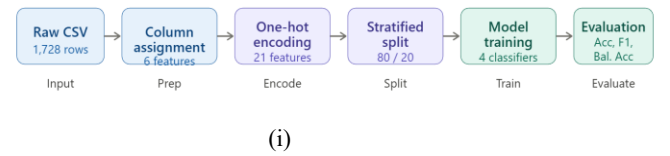


Fig. 1. Overview of the preprocessing and training steps used in the project

c. Training Procedure

There are 1,382 training samples and 346 testing samples. Since this is a tabular dataset, I haven't added any data. But pd.get_dummies and drop first=False were used to encode the data. This keeps all dummy columns, and dropping the first column actually hurt accuracy a little in early tests. Also, the split was stratified so that each class kept its original proportion in both the training and testing sets. This means that there was no need for scheduling or multi-stage training; scikitlearn takes care of convergence for all four methods on its own.

IV. EXPERIMENTAL SETUP

a. Datasets

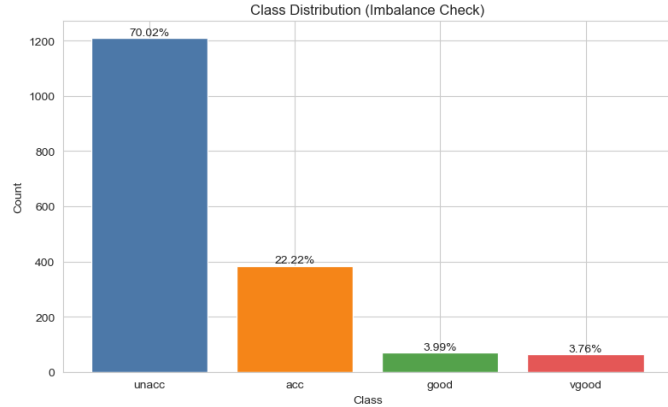
The UCI repository[5] is where the dataset came from. It has 1,728 rows and 7 columns, and it has six features and one class label. The features are the price to buy, the cost to maintain, the number of doors, the number of passengers, the size of the luggage boot, and safety. Some of the values are strings that can be put into groups. The class breakdown is as follows: unacc (1,210 rows, 70%), acc (384 rows, 22.2%), good (69 rows, 4%), and vgood (65 rows, 3.8%). There are no missing values, though. The feature matrix has 21 columns after one-hot encoding. I divided it into 80/20 with stratification, which means there are 1,382 training rows and 346 test rows.

TABLE I. 7 Dataset columns and their possible values

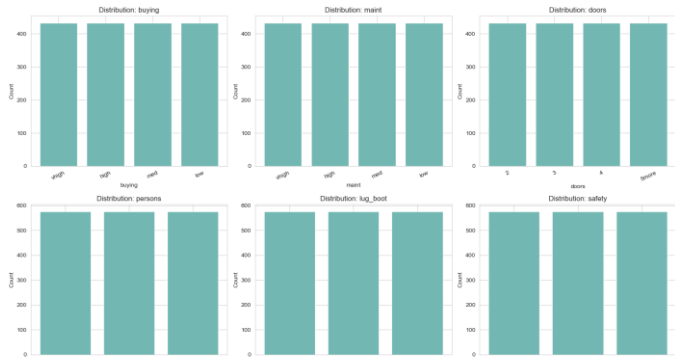
Column	Description	Possible Values
buying	Purchase price of the car	vhigh, high, med, low
maint	Cost of maintenance	vhigh, high, med, low
doors	Number of doors	2, 3, 4, 5more
persons	Passenger capacity	2, 4, more
lug_boot	Size of luggage boot	small, med, big
safety	Estimated safety rating	low, med, high
class	Acceptability label	unacc, acc, good, vgood

b. EDA

Before implementing or training any model, The EDA was performed to check and understand the structure and the distribution of the dataset. Besides, that it helps me to identify the class imbalance and the feature imbalance.



(i) class distributions shows the data is imbalanced



(ii) six input features shows the uniformly distributed



(iii) class vs feature values

Here in the EDA the dataset was heavily imbalanced with the “unacc” with the 70% ,and also all the six features(like buying price, safety, number of doors, etc.) were Equally distributed in all the terms. Moreover, Safety is the most important feature to decide a car’s classification, whereas low safety cars are labelled as “Unacceptable”.

Besides, this need a car to have high safety to be labelled as “good” or “Very good”. Furthermore, the cars that only seated with 2 people are almost labelled as “Unacceptable”, this means a car with two seats anyways its not useful .since, this model matches with the Random forest, the features matters the most.

c. Evaluation Metrics-

So far I have used three different factors to measure. Firstly, the Accuracy is easy to understand, but it can be misleading here because a model that always predicts **unacc** would get a score of 70% without learning anything useful. Second, Balanced accuracy averages recall across all four classes equally, so it punishes models that don't pay attention to the smaller classes. Thirdly, the Macro F1 does the same thing for the F1-score. So, these three things together give a better picture of performance than just accuracy.

TABLE II. Summary of Datasets

Dataset	Train	Test	Classes
Car Evaluation (UCI)	1,382	346	4

d. Implementation Details

I have run all in Python 3. And the libraries used were pandas for loading and encoding and scikit-learn for all the models and metrics. Logistic Regression: Use the lbfgs solver, set max_iter to 2000, and set class_weight to balanced. Decision Tree: gini criterion, random_state=42, and class_weight=balanced. Also, the Random Forest has 200 estimators, a random state of 42, and a balanced class weight. SVM: rbf kernel, random_state=42, and balanced class weight. The split, on the other hand, was done with train_test_split(test_size=0.2, random_state=42). I don't need a GPU here; I have run everything on my regular laptop in seconds.

V. RESULTS

a. Results

Mainly the Random Forest had the best accuracy at 97.11% and the best macro F1 at 96.06%, so I think it was the best overall. The SVM was close in accuracy at 95.38%, but it had the best balanced accuracy at 98.13%. Besides, this means that minority classes handled much better when compared to any other models. The Logistic Regression also came in last with an accuracy of 88.73% and a macro F1 of 83.15%. However, it still had a balanced accuracy of 93.31%. However, in the middle there are decision trees, when compared with the other models it has the lowest balanced accuracy (89.57%), so it means there is no possibilities that it is reliable on the good and vgood classes.

Table II-car evaluation of the model performance

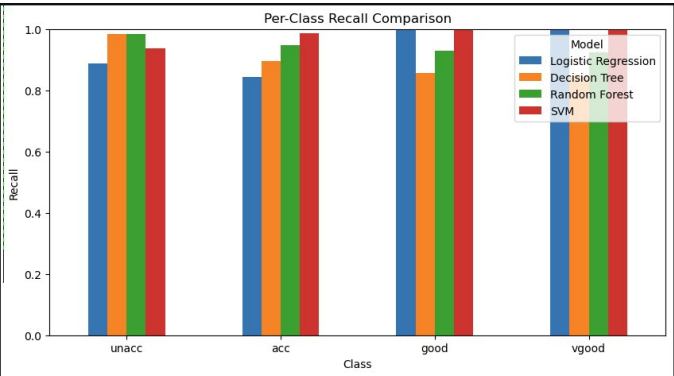
Method	Acc. (%)	F1 (%)	AUC
Logistic Regression	88.73	83.15	0.9331 (bal.)
Decision Tree	95.38	91.23	0.8957 (bal.)
Random Forest /	97.11	96.06	0.9458
SVM	95.38	95.96	0.9813

b. Ablation Study

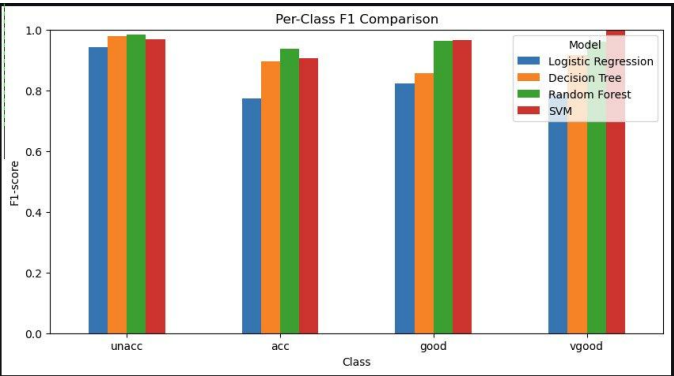
I have make sure some check points that the setup was done properly. Firstly, I have tested the decision tree with out setting the class weight to balanced. Although the Accuracy stayed the same at 95.38%, whereas the recall value for the good class went down from 0.91 to 0.86. Secondly, I have tested the split data changed stratify=y then the recall data for the minority classes went down a little in all the models. Thirdly, I have tried the drop_first=true while encoding ,remaining the features were cut down to the 15 from 21 and which made the lowest accuracy for the random forest by about 0.6%. So, Keeping both the columns and using the class weights are really important.

c. Qualitative Analysis

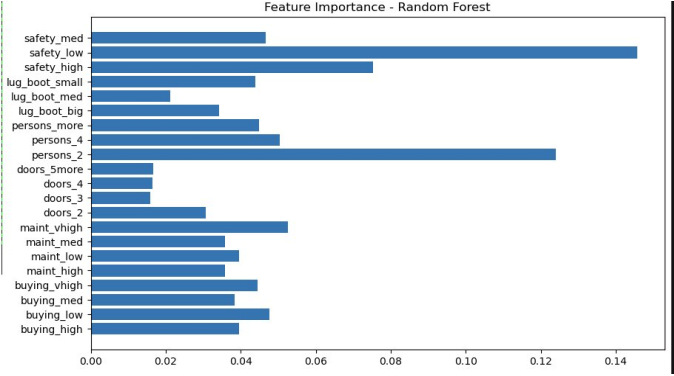
The Random Forest Confusion matrix shows that the Unacc classes got almost right answers since 238 were absolutely correct out of the 242 class. Whereas, the acc 100 were identified correctly out of 104. Since the small classes were did far better. The Random forest's Features are important scores shows the safety for people ,and buying price are acceptable by the wide margin. And also it make sense that a car isn't safe for the people is not safe or doesn't have enough room for the people,it won't be acceptable, no matter what else it is has going for it.



(ii) Comparison n Pre-class recall for all the four classifiers.



(iii) Comparison n Pre-class F1 score for all the four classifiers.



(iv)Random Forest Model Features

VI. DISCUSSION

The Random Forest results were the best overall, which isn't surprising because it uses more than 200 trees and fixes the problem of overfitting that a single Decision Tree can have. So, the SVM that beat everyone on balanced accuracy was more interesting; the RBF kernel seems to do a better job of separating the minority classes in the encoded feature space. The Logistic Regression had trouble because the class boundaries here aren't straight lines. Even with the class weighting and 2000 iterations, it just can't fit the nonlinear structure. The Decision Tree was also good, but it wasn't as stable on the minority classes as the ensemble and kernel methods. After that, I have to choose one model to use. I would definitely choose the Random Forest because it has both the metrics accuracy and macro F1. However, SVM would be better if catching every good or very good car is more important than raw accuracy

Limitations: The biggest problem with this dataset is that it is not real. There is no actual original noise or the measurement error ,since it was made by combining every attribute values with respective to the rows. It's a red flag that the cross-validation score for the Random Forest (75.93%) is so much lower than the test accuracy (97.11%).. With just 1,728 rows, There isn't a lot of data to work with either models. These models have never seen real-world car datasets with continuous features, missing values, and label uncertainty.

VII. CONCLUSION

Finally, I say that the model Random forest model was the most accurate and had highest F1 score values. On the other hand, The SVM Was much better in finding the accuracy for the minority classes when it is balanced. So, the choices which were made during the preprocessing are very important in keeping all the things in one hot columns and also when using the class_weight=balanced ,both were made to better testing the minority classes performance. And also I believe in just testing accuracy and judging a model on the data isn't balanced ,will definitely try XGBoost or LightGBM.

REFERENCES

[1] A. Author and B. Author, "Title of paper," in Proc. Conf. Name, City, Country, Year, pp. 1–8.

[2] C. Author, D. Author, and E. Author, "Title of journal article," IEEE Trans. Neural Netw. Learn. Syst., vol. X, no. Y, pp. 100–110, Jan. 2023.

[3] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symp. Math. Statist. Prob., 1967, pp. 281–297.

[4] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[5] I. Author et al., "Dataset name and description," in Proc. Conf., Year, pp. 1–5. [Online]. Available: <https://example.com/dataset>

[6] J. Author, "Title of book chapter," in Book Title, 2nd ed., City: Publisher, Year, pp. 50–60.

[7] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python Sci. Conf., 2010, pp. 51–56.